Software Measurement and Effort Estimation

Lecture 8

Agenda

- Software Measurement
- Effort Estimation

Software Measurement

Software Measurement

- Although the terms measure, measurement, and metrics are often used interchangeably, it is important to note the subtle differences between them.
- Measure provides a quantitative indication of the extent, amount, dimension, capacity, or size of some attribute of a product or process.
- Measurement is the act of determining a measure.
- According to the IEEE Standard Glossary of Software Engineering Terminology, a metric is defined as “a quantitative measure of the degree to which a system, component, or process possesses a given attribute”.

Software Measurement

- In general, if you don’t measure, judgment can be based only on subjective evaluation.
- With measurement, trends (either good or bad) can be spotted, better estimates can be made, and true improvement can be accomplished over time.
- “What gets measured gets managed”
  - Peter Drucker

Measurement Principles

- A metric should have desirable mathematical properties.
  - That is, the metric's value should be in a meaningful range (e.g., 0 to 1, where 0 truly means absence, 1 indicates the maximum value, and 0.5 represents the "halfway point").
- When a metric represents a software characteristic that increases when positive traits occur or decreases when undesirable traits are encountered, the value of the metric should increase or decrease in the same manner.
- Each metric should be validated empirically in a wide variety of contexts before being published or used to make decisions.
  - A metric should measure the factor of interest, independently of other factors. It should "scale up" to large systems and work in a variety of programming languages and system domains.

Software Measurement

- Project metrics enable a software project manager to
  - (1) assess the status of an ongoing project,
  - (2) track potential risks,
  - (3) uncover problem areas before they go “critical,”
  - (4) adjust workflow or tasks, and
  - (5) evaluate the project team’s ability to control quality of software work products.
- Software measures are often normalized using either size- or function-oriented metrics.

Size-Oriented Metrics

Size-Oriented Metrics

- Size-oriented software metrics are derived by normalizing quality and/or productivity measures by considering the size of the software that has been produced.
- If a software organization maintains simple records, a table of size-oriented measures can be created.

Size-Oriented Metrics

- The table lists each software development project that has been completed over the past few years and corresponding measures for that project.
- Referring to the table entry for project alpha : 12,100 lines of code were developed with 24 person-months of effort at a cost of $168,000.

FIGURE 32.2

Size-oriented metrics

Project	LOC	Effort	$(000)	Pp. doc.	Errors	Defects	People
alpha	12,100	24	168	365	134	29	3
beta	27,200	62	440	1224	321	86	5
gamma	20,200	43	314	1050	256	64	6
⋮	⋮	⋮	⋮	⋮	⋮		

Size-Oriented Metrics

- A set of simple size-oriented metrics can be developed for each project:
  - Errors per KLOC (thousand lines of code)
  - Defects per KLOC
  - $ per KLOC
  - Pages of documentation per KLOC
- In addition, other interesting metrics can be computed:
  - Errors per person-month
  - KLOC per person-month
  - $ per page of documentation

Size-Oriented Metrics

- Proponents of the LOC measure claim that LOC is an “artifact” of all software development projects that can be easily counted, that many existing software estimation models use LOC or KLOC as a key input, and that a large body of literature and data predicated on LOC already exists.
- On the other hand, opponents argue that LOC measures are programming language dependent, that when productivity is considered, they penalize well-designed but shorter programs; that they cannot easily accommodate nonprocedural languages; and that their use in estimation requires a level of detail that may be difficult to achieve
  - i.e., the planner must estimate the LOC to be produced long before analysis and design have been completed

Function-Oriented Metrics

Function-Oriented Metrics

- Function-oriented software metrics use a measure of the functionality delivered by the application as a normalization value.
- The most widely used function-oriented metric is the function point (FP).
- Computation of the function point is based on characteristics of the software's information domain and complexity

Function-Oriented Metrics

- The function point, like the LOC measure, is controversial.
- Proponents claim that FP is programming language–independent, making it ideal for applications using conventional and nonprocedural languages, and that it is based on data that are more likely to be known early in the evolution of a project, making FP more attractive as an estimation approach.
- Opponents claim that the method requires some "sleight of hand" in that computation is based on subjective rather than objective data, that counts of the information domain (and other dimensions) can be difficult to collect after the fact, and that FP has no direct physical meaning—it's just a number.

The Function Point (FP) Metric

- The function point (FP) metric can be used effectively as a means for measuring the functionality delivered by a system.
- Using historical data, the FP metric can then be used to
  - (1) estimate the cost or effort required to design, code, and test the software;
  - (2) predict the number of errors that will be encountered during testing; and
  - (3) forecast the number of components and/or the number of projected source lines in the implemented system.

The Function Point (FP) Metric

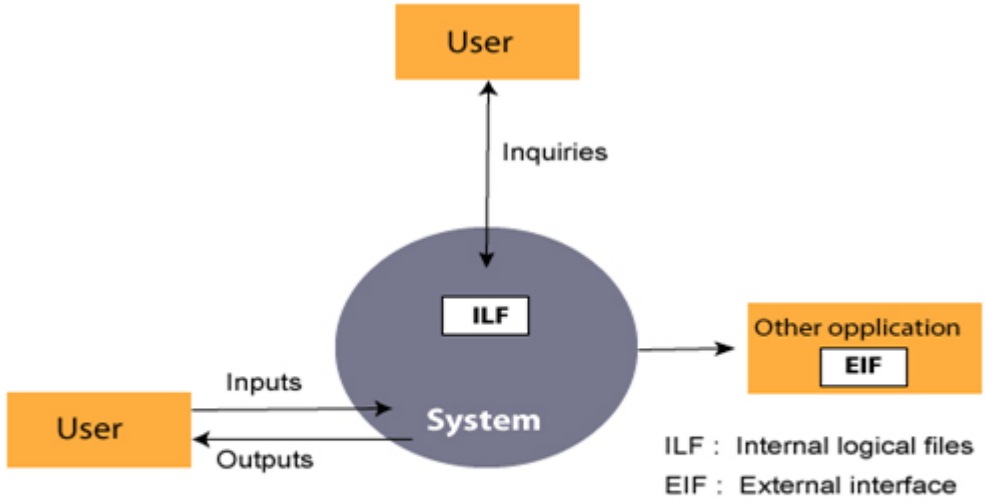
- Function points are derived using an empirical relationship based on countable (direct) measures of software's information domain and qualitative assessments of software complexity. Information domain values are defined in the following manner:

FIGURE 30.1

Computing function points

Information Domain Value	Count		Weighting factor Simple	Average	Complex		
External Inputs (EIs)		3	3	4	6	=	
External Outputs (EOs)		3	4	5	7	=	
External Inquiries (EQs)		3	3	4	6	=	
Internal Logical Files (ILFs)		3	7	10	15	=	
External Interface Files (EIFs)		3	5	7	10	=	
Count total	→						

The Function Point (FP) Metric



FPAs Functional Units System

Courtesy: https://www.javatpoint.com/software-engineering-functional-point-fp-analysis

The Function Point (FP) Metric

- Number of external inputs (EIs).
  - Each external input originates from a user or is transmitted from another application and provides distinct application-oriented data or control information. Inputs are often used to update internal logical files (ILFs). Inputs should be distinguished from inquiries, which are counted separately.
- Number of external outputs (EOs).
  - Each external output is derived data within the application that provides information to the user. In this context, external output refers to reports, screens, error messages, and the like. Individual data items within a report are not counted separately.

The Function Point (FP) Metric

- Number of external inquiries (EQs).
  - An external inquiry is defined as an online input that results in the generation of some immediate software response in the form of an online output (often retrieved from an ILF).
- Number of internal logical files (ILFs).
  - Each internal logical file is a logical grouping of data that resides within the application's boundary and is maintained via external inputs.
- Number of external interface files (EIFs).
  - Each external interface file is a logical grouping of data that resides external to the application but provides data that may be of use to the application.

The Function Point (FP) Metric

- To compute function points (FP), the following relationship is used:

$$\text{FP} = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)]$$

- where count total is the sum of all FP entries obtained from
- The F_i (i=1 to 14) are value adjustment factors (VAF) based on responses to a set of questions (shown in the next slide).
  - Value adjustment factors are used to provide an indication of problem complexity.
- Each of these questions is answered using an ordinal scale that ranges from 0 (not important or applicable) to 5 (absolutely essential).
- The constant values in Equation and the weighting factors that are applied to information domain counts are determined empirically

The Function Point (FP) Metric: VAF Questions

1. Does the system require reliable backup and recovery?
2. Are specialized data communications required to transfer information to or from the application?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require online data entry?
7. Does the online data entry require the input transaction to be built over multiple screens or operations?
8. Are the ILFs updated online?

The Function Point (FP) Metric: VAF Questions

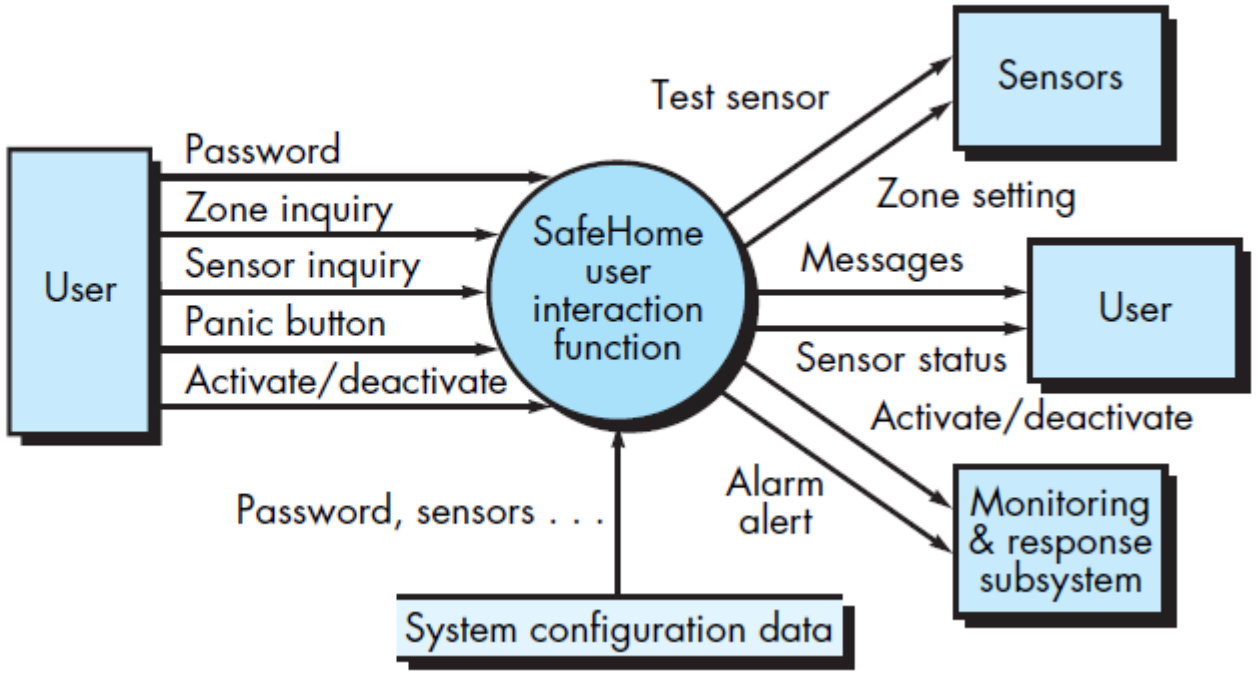
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

The Function Point (FP) Metric: An example

- Let us see an example…

FIGURE 30.2

A flow model for *SafeHome* user interaction function



The Function Point (FP) Metric: An example

- The flow diagram is evaluated to determine a set of key information domain measures required for computation of the function point metric.
- Three external inputs— password, panic button, and activate/deactivate —are shown in the figure along with
- two external inquiries— zone inquiry and sensor inquiry .
- One ILF (system configuration file) is shown.
- Two external outputs (messages and sensor status) and
- four EIFs (test sensor, zone setting, activate/deactivate, and alarm alert) are also present

The Function Point (FP) Metric: An example

- For the purposes of this example, we assume that $\Sigma (F_i)$ is 46 (a moderately complex product).
- Therefore, $\mathrm{FP} = 50 \times [0.65 + (0.01 \times 46)] = 56$

FIGURE 30.3 Computing function points

Information Domain Value	Count		Weighting factor: Simple	Average	Complex		
External Inputs (EIs)	3	3	(3)	4	6	=	9
External Outputs (EOs)	2	3	(4)	5	7	=	8
External Inquiries (EQs)	2	3	(3)	4	6	=	6
Internal Logical Files (ILFs)	1	3	(7)	10	15	=	7
External Interface Files (EIFs)	4	3	(5)	7	10	=	20
Count total						→	50

Reconciling LOC and FP Metrics

Reconciling LOC and FP Metrics

- The relationship between lines of code and function points depends upon the programming language that is used to implement the software and the quality of the design.
- A number of studies have attempted to relate FP and LOC measures.

Reconciling LOC and FP Metrics

Programming Language	LOC per Function Point			
	Avg.	Median	Low	High
Ada	154	—	104	205
ASP	56	50	32	106
Assembler	337	315	91	694
C	148	107	22	704
C++	59	53	20	178
C#	58	59	51	704
COBOL	80	78	8	400
ColdFusion	68	56	52	105
DBase IV	52	—	—	—
Easytrieve+	33	34	25	41
Focus	43	42	32	56
FORTRAN	90	118	35	—
FoxPro	32	35	25	35
HTML	43	42	35	53
Informix	42	31	24	57
J2EE	57	50	50	67
Java	55	53	9	214
JavaScript	54	55	45	63
JSP	59	—	—	—
Lotus Notes	23	21	15	46

Programming Language	LOC per Function Point			
	Avg.	Median	Low	High
Mantis	71	27	22	250
Natural	51	53	34	60
.NET	60	60	60	60
Oracle	42	29	12	217
OracleDev2K	35	30	23	100
PeopleSoft	37	32	34	40
Perl	57	57	45	60
PL/1	58	57	27	92
Powerbuilder	28	22	8	105
RPG II/III	61	49	24	155
SAS	50	35	33	49
Smalltalk	26	19	10	55
SQL	31	37	13	80
VBScript	38	37	29	50
Visual Basic	50	52	14	276

Object-Oriented Metrics

Object-Oriented Metrics

- Conventional software project metrics (LOC or FP) can be used to estimate object-oriented software projects.
- However, these metrics do not provide enough granularity for the schedule and effort adjustments that are required as you iterate through an evolutionary or incremental process.
- Lorenz and Kidd suggest the following set of metrics for OO projects:
  - Number of scenario scripts.
  - Number of key classes.
  - Number of support classes.
  - Average number of support classes per key class.
  - Number of subsystems.

Use Case-Oriented Metrics

Use Case-Oriented Metrics

- Use cases are used widely as a method for describing customer-level or business domain requirements that imply software features and functions.
- It would seem reasonable to use the use case as a normalization measure similar to LOC or FP.
- Like FP, the use case is defined early in the software process, allowing it to be used for estimation before significant modelling and construction activities are initiated.

WebApp Project Metrics

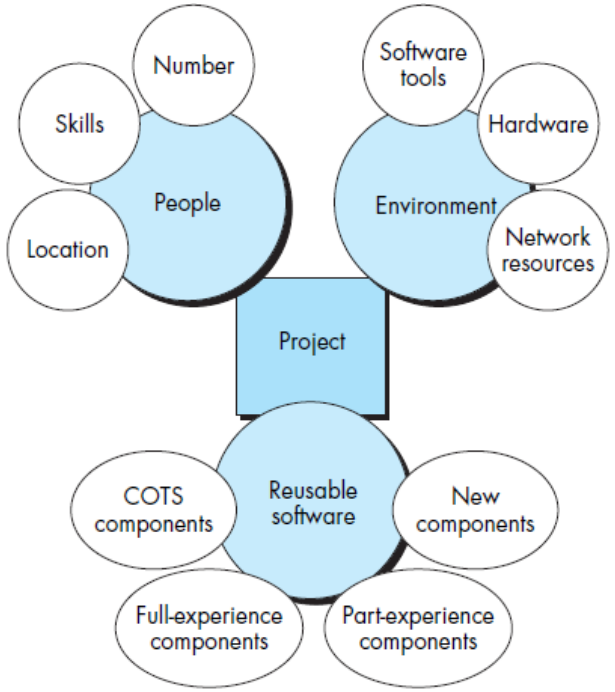
WebApp Project Metrics

- The objective of all WebApp projects is to deliver a combination of content and functionality to the end user.
- Measures and metrics used for traditional software engineering projects are difficult to translate directly to WebApps. Measures that can be collected are the following:
  - Number of static Web pages.
  - Number of dynamic Web pages.
  - Number of internal page links.
  - Number of persistent data objects.
  - Number of external systems interfaced.
  - Number of static content objects.
  - Number of dynamic content objects.
  - Number of executable functions (e.g., a script or applet).

Effort Estimation

Effort Estimation

- A key planning task is the estimation of the resources required to accomplish the software development effort.
  - Three major categories of software engineering resources—people, reusable software components, and the development environment (hardware and software tools).



Effort Estimation

- Software cost and effort estimation will never be an exact science.
- Too many variables—human, technical, environmental, political—can affect the ultimate cost of software and effort applied to develop it.
- However, software project estimation can be transformed from a black art to a series of systematic steps that provide estimates with acceptable risk. To achieve reliable cost and effort estimates, a number of options arise:
  - Delay estimation until late in the project (obviously, we can achieve 100 percent accurate estimates after the project is complete!). - Impractical
  - Base estimates on similar projects that have already been completed. - Unfortunately, past experience has not always been a good indicator of future results.
  - Use relatively simple decomposition techniques to generate project cost and effort estimates.
  - Use one or more empirical models for software cost and effort estimation.

Effort Estimation

- Ideally, the techniques noted for each option should be applied in tandem; each used as a cross-check for the other.
- Decomposition techniques take a divide-and-conquer approach to software project estimation. By decomposing a project into major functions and related software engineering activities, cost and effort estimation can be performed in a stepwise fashion.
- Empirical estimation models can be used to complement decomposition techniques and offer a potentially valuable estimation approach in their own right.

Decomposition Techniques

Decomposition Techniques

- Software project estimation is a form of problem solving, and in most cases, the problem to be solved (i.e., developing a cost and effort estimate for a software project) is too complex to be considered in one piece.
- For this reason, you should decompose the problem, recharacterizing it as a set of smaller (and hopefully, more manageable) problems.
- Software sizing represents your first major challenge as a planner

Decomposition Techniques

- When LOC is used as the estimation variable, decomposition is absolutely essential and is often taken to considerable levels of detail.
  - The greater the degree of partitioning, the more likely reasonably accurate estimates of LOC can be developed.
- For FP estimates, decomposition works differently. Rather than focusing on function, each of the information domain characteristics—inputs, outputs, data files, inquiries, and external interfaces—as well as the 14 complexity adjustment values are estimated.
  - The resultant estimates can then be used to derive an FP value that can be tied to past data and used to generate an estimate.

An Example of LOC-Based Estimation

FIGURE 33.2

Estimation table for the LOC methods

Function	Estimated LOC
User interface and control facilities (UICF)	2,300
Two-dimensional geometric analysis (2DGA)	5,300
Three-dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (CGDF)	4,950
Peripheral control function (PCF)	2,100
Design analysis modules (DAM)	8,400
Estimated lines of code	*33,200*

An Example of FP-Based Estimation

Factor	Value
Backup and recovery	4
Data communications	2
Distributed processing	0
Performance critical	4
Existing operating environment	3
Online data entry	4
Input transaction over multiple screens	5
Master files updated online	3
Information domain values complex	5
Internal processing complex	5
Code designed for reuse	4
Conversion/installation in design	3
Multiple installations	5
Application designed for change	5
Value adjustment factor	**1.17**

An Example of FP-Based Estimation

FIGURE 33.3

Estimating information domain values

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	97
Number of external outputs	12	15	22	16	5	78
Number of external inquiries	16	22	28	22	5	88
Number of internal logical files	4	4	5	4	10	42
Number of external interface files	2	2	3	2	7	15
Count total						*320*

Finally, the estimated number of FP is derived:

$$FP_{estimated} = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)] = 375$$

Empirical Estimation Models

Empirical Estimation Models

- An estimation model for computer software uses empirically derived formulas to predict effort as a function of LOC or FP.
- Values for LOC or FP are estimated using the approach described in the previous slides.
- But instead of using the tables described in those slides, the resultant values for LOC or FP are plugged into the estimation model.
- No estimation model is appropriate for all classes of software and in all development environments. Therefore, you should use the results obtained from such models judiciously.

Empirical Estimation Models

- A typical estimation model is derived using regression analysis on data collected from past software projects.
- The overall structure of such models takes the form

$$E = A + B \times (e_v)^C$$

- where A, B, and C are empirically derived constants, E is effort in person-months, and e_v is the estimation variable (either LOC or FP).

The COCOMO II Model

- In his classic book on software engineering economics, Barry Boehm [Boe81] introduced a hierarchy of software estimation models bearing the name COCOMO, for COnstructive COst MOdel.
- The original COCOMO model became one of the most widely used and discussed software cost estimation models in the industry. It has evolved into a more comprehensive estimation model, called COCOMO II.
- Like its predecessor, COCOMO II is actually a hierarchy of estimation models that address different “stages” of the software process.
- Like all estimation models for software, the COCOMO II models require sizing information.
- Three different sizing options are available as part of the model hierarchy: object points, function points, and lines of source code.
  - An object point is an indirect software measure that is computed using counts of the number of (1) screens (at the user interface), (2) reports, and (3) components likely to be required to build the application, along with complexity factors.

Next On

Take Away Points

- What are size-oriented metrics?
- What are function-oriented metrics?
- How do we calculate the Function Point (FP) metric?
- Other metrics: Object-Oriented, WebApp, etc.
- What is difference between software sizing and its effort estimation?
- How are decomposition techniques and empirical estimation models intertwined?
- What is COCOMO II?

Further Reading

- Pressman and Maxim, “Software Engineering”
  - Chapter 32: Process and Project Metrics
  - Chapter 30: Product Metrics
  - Chapter 33: Estimation For Software Projects
- Ian Somerville, “Software Engineering”
  - Chapter 23: Project Planning
- Java T Point, “Software Metrics”
  - https://www.javatpoint.com/software-engineering-software-metrics

Next Lecture

- Unit 2: Risk Management
- Unit 3: Tools for Monitoring Software Development Projects.